Repairing Privately Generated Synthetic Data with Integrity and Statistical Constraints

Itay Chairman
Hebrew University
itay.chairman@mail.huji.ac.il

Xi He
University of Waterloo
xi.he@uwaterloo.ca

Amir Gilad
Hebrew University
amirg@cs.huji.ac.il

ABSTRACT

Differentially private synthetic data enables privacy-preserving data analysis by allowing unrestricted downstream computation under rigorous privacy guarantees. Most existing synthesis methods focus on preserving statistical properties of the private data, typically via noisy low dimensional marginals, and recent systems have begun to incorporate integrity constraints such as Denial Constraints (DCs). An important yet unexplored challenge arises when a DP synthetic dataset has already been generated, and is later found to violate integrity constraints or deviate from key statistical signals. Regenerating such data is expensive and not always possible, whereas existing data repair techniques are not designed to account for the specific statistical issues underlying DP synthesis.

We formalize this challenge as the *synthetic data repair problem*, which seeks to modify an existing synthetic dataset so that it satisfies a set of DCs while ensuring the satisfaction of specific representative statistics obtained from the private data. We show that the problem is NP-hard even without DCs, highlighting the inherent difficulty of jointly enforcing statistical and logical constraints. We propose a two-phase framework that privately selects informative statistic constraints, followed by either a repair step implemented by an optimal Integer Linear Program-based repair or a heuristic weighted vertex cover-based repair. Experiments on four real-world datasets demonstrate that our approach enforces DC consistency while preserving statistical fidelity, providing effective tradeoffs between accuracy and scalability.

1 INTRODUCTION

Differential privacy (DP) is widely regarded as the gold standard for protecting individual privacy, offering strong, quantifiable, and composable guarantees for data analysis and release [16, 19]. A particularly powerful application of DP is synthetic data generation. By virtue of the post-processing property of DP [19], DP synthetic datasets can be freely shared and analyzed without additional privacy loss, supporting tasks such as exploratory analysis, model training, and query answering over sensitive data. A large body of work has developed DP synthetic data generation frameworks based on diverse techniques, ranging from deep generative models [55, 58, 59, 63] to statistical modeling approaches [26, 27, 40, 41, 64]. Despite their differences, these methods share a common goal of preserving the statistical characteristics of the private data, most notably through the use of noisy low dimensional marginals, typically two-way marginals, to guide synthetic data generation [40, 42, 64].

In addition to statistical fidelity, synthetic data may be required to satisfy integrity constraints that encode domain knowledge and semantic validity. Denial Constraints (DCs) [8, 53] form the most general and widely adopted class of such constraints, subsuming functional dependencies and other common restrictions by forbidding disallowed patterns across tuples. Recent work has begun to address this need, most notably through Kamino [23], which generates DP synthetic data that satisfies a specified set of DCs.

However, existing systems such as Kamino focus exclusively on the *generation* of synthetic data from scratch. In many realistic settings, synthetic data may already have been generated, often at substantial computational cost, and only later discovered to be inconsistent with newly imposed or previously overlooked integrity constraints. At the same time, synthetic data is increasingly expected to satisfy additional *statistical* constraints aimed at mitigating bias by preserving specific properties. This requirement is particularly salient in privacy-preserving settings, as the DP synthesis process may preserve or even amplify undesirable biases present in the original data [22]. To date, there is no solution tailored to repairing already generated DP synthetic data in a way that jointly addresses logical consistency and statistical considerations.

A straightforward response to inconsistency or bias is to regenerate the synthetic data, but this approach is often prohibitively expensive due to the computational complexity of modern DP synthesis pipelines. An alternative is to apply existing data repair techniques [2, 25, 37, 53]. However, classical data repair focuses on enforcing integrity constraints and disregards the statistical objectives that motivated synthetic data generation in the first place, potentially yielding repaired datasets that remain biased or diverge from the distribution of the private data. Recently, Liu et al. [34] studied data repair under functional dependencies with a single representation constraint, emphasizing the importance of combining statistical considerations in data repair for bias mitigation. While promising, this work neither considers richer 2-way marginal constraints (2-MCs), used for correlation and bias measurement, nor addresses the challenges common in DP data synthesis.

Motivated by these gaps, we propose a framework specifically designed for *synthetic data repair under DP*. We formalize a novel problem in which the input consists of an existing synthetic dataset and a set of DCs. The goal is to privately generate a collection of noisy 2-way marginal constraints (2-MCs) intended to control bias and then repair the dataset by deleting tuples so that the resulting synthetic data simultaneously satisfies the DCs and adheres to the 2-MCs. While DCs enforce logical validity, the noisy 2-MCs encode statistical properties of the original private data and serve as a mechanism for bias control. Although data repair under DCs is known to be computationally hard [37], we show that the synthetic data repair problem is NP-hard even in the absence of DCs.

We address this challenge through a two-step approach. First, we privately obtain relevant 2-MCs using the one-shot top-k-mechanism [14]. Second, we repair the synthetic dataset using one of two complementary approaches. The first is an ILP-based formulation that extends recent work on cost-aware subset repairs [34] and yields optimal solutions that guarantee the satisfaction of both the 2-MCs and DCs, albeit with limited practical use as its repairs may be too drastic in terms of tuple deletions. The second is a greedy algorithm based on the weighted vertex cover formulation over the conflict graph [10], augmented with a novel dynamic weight function tailored to 2-MC satisfaction. While this approach may be slower and does not guarantee full 2-MC satisfaction, it strikes a middle-ground between satisfying the DCs, the 2-MCs, and deletion ratio.

We evaluate our framework through an extensive experimental study on four real-world datasets. The results demonstrate that our approach efficiently identifies meaningful marginal constraints and that both the ILP and vertex-cover-based methods guarantee DC satisfaction while producing repairs that remain faithful to the private data. In particular, our weighted vertex cover approach achieves a compelling balance between integrity, bias control, and scalability, making it a practical solution for post-hoc repair of privately generated synthetic data.

Table 1: DC and 2-MC violations for synthetic vs. repaired data for different synthesizers using the Adult dataset. Darker shades of red indicates a higher violation rate.

Synthesizer	Synthetic Data		Repaired Synth. Data	
	DCs Viol.	% 2-MCs Viol.	DCs Viol.	% 2-MCs Viol.
AIM	875, 565	45%	0	27%
MST	710, 000	50%	0	12.5%
PATE-CTGAN	10, 619, 261	72%	0	25%

Example 1.1. *Consider the Adult dataset [31], which contains sensitive attributes such as income, sex, race, and education. We use a sample of* 10,000 *tuples that satisfies several inherent DCs, for example, that two individuals cannot share the same years of schooling while having different education levels. Suppose an analyst wishes to draw conclusions from a previously generated DP synthetic dataset of size* 10,000*. To demonstrate this, we produced synthetic data using three state-of-the-art synthesizers with privacy budget* $\epsilon = 10$*: AIM [41], MST [40], and PATE-CTGAN [63].*

Prior work [23] showed that DC violations significantly reduce the reliability and usability of synthetic data. At the same time, the analyst seeks to preserve key distributional properties relevant to fairness, such as minimum thresholds on the joint distributions of (sex, income) and (education, income), e.g., ensuring that at least 14% *of the data correspond to African American females. Such sensitive marginals are particularly important in DP settings, where noise may amplify existing biases [22]. We therefore derive* 10 *noisy two-way marginal constraints (2-MCs) from the private data.*

Table 1 reports the number of DC violations and the percentage of violated 2-MCs for each synthesizer, averaged over 4 *runs. All methods generate synthetic datasets containing substantial DC violations. Moreover, the percentage of violated 2-MCs is high across all synthesizers, indicating poor adherence to fairness-related marginals. On the other hand, our greedy repair approach addresses both issues simultaneously. It reduces DC violations to* 0 *for all synthesizers and significantly lowers 2-MC violations, while deleting only a portion of the data to preserve overall utility. Specifically, the repair removes* 19% *of the tuples for MST,* 44% *for AIM, and* 86% *for PATE-CTGAN (due to the extreme violation rate of the data).*

This example highlights that state-of-the-art DP synthesizers do not guarantee logical consistency or adherence to critical marginal properties. When re-generating the data is unhelpful or impractical, synthetic data repair can improve existing DP synthetic datasets by enforcing DC compliance and substantially reducing 2-MC violations, thereby enhancing reliability and adherence to relevant statistics.

Contributions. This work makes the following contributions:

- **Problem formulation (Section 3).** We introduce and formalize the *synthetic data repair under DP* problem, the first framework that addresses post-hoc repair of already generated DP synthetic data while jointly enforcing DCs and statistical constraints aimed at bias control. We show that the problem is NP-hard even in the absence of integrity constraints.
- **Repair algorithms (Section 4).** We develop two complementary repair techniques: an ILP-based approach that extends recent work [34] to support DCs and 2-MCs, and yields optimal solutions, and a greedy dynamic weighted vertex cover algorithm.
- **Private marginal acquisition (Section 5).** We propose a DP procedure for identifying and estimating informative marginal constraints based on data generation approaches [40, 41].
- **Experimental evaluation (Section 6).** We conduct an extensive empirical study on four real-world datasets, demonstrating the tradeoffs between repair quality and scalability. We show that our weighted vertex cover approach strikes a practical balance between integrity, statistical fidelity, and performance. We further show that our approach generates high quality repairs, even when the 2-MCs are generated with limited privacy budget.

We review the necessary background for the paper in Section 2, discuss related work in Section 7, and conclude in Section 8.

2 PRELIMINARIES

We next review the necessary definitions from previous work. Notations are summarized in Table 2.

ID	Edu-num	Edu.	Occu.	Income	Age	Race
t_1	14	Undergrad	Manager	50K	52	AA
t_2	13	HS	Craft-repair	47K	48	SA
t_3	14	Undergrad	Farming	50K	25	C
t_4	22	Prof_school	Prof	62K	57	AA

(a) Private dataset D_p^E.

ID	Edu-num	Edu.	Occu.	Income	Age	Race
t_1	15	Undergrad	Manager	50K	55	AA
t_2	12	HS	Craft-repair	40K	45	SA
t_3	15	Undergrad	Farming	50K	22	C
t_4	22	Prof_school	Prof	60K	55	AA
t_5	14	Undergrad	Prof	30K	50	AA
t_6	16	Undergrad	Craft-repair	60K	50	AA

(b) Synthetic dataset D_s^E.

Figure 1: Private and synthetic databases based on population datasets [9, 21, 31].

2.1 Databases and Repairs

We consider a single-relation schema $\mathcal{A}ttr = (A_1, \ldots, A_m)$, which is a vector of distinct attribute names A_i, each associated with a

Table 2: Notation.

Symbol	Meaning
$\mathcal{A}ttr$	Attribute set
D_p	Private dataset
D_s	Synthetic dataset
D_r	Repaired dataset
$\sigma \in \Sigma$	Denial Constraint (DC)
$m_D(i, j)$	Marginal on D
$m_D(i, j, \tau) \in \mathcal{M}$	2-way Marginal Constraint on D (2-MC)
$G = (V, E, w)$	Weighted conflict graph
u	Utility function for the exponential mechanism

domain $Dom(A_i)$ of values. A database D over $\mathcal{A}ttr$ is associated with a set $tids(D)$ of *tuple identifiers*, and it maps every identifier $i \in tids(D)$ to a tuple $D[i] = (a_1, \ldots, a_m)$ in $A_1 \times \cdots \times A_m$. A database D' is a *subset* of D, denoted $D' \subseteq D$, if D' is obtained from D by deleting zero or more tuples, that is, $tids(D') \subseteq tids(D)$ and $D'[i] = D[i]$ for all $i \in tids(D')$. The paper will consider a synthetic database D_s generated from a private dataset D_p (protected by Differential Privacy, see Section 2.2). D_s and D_p are assumed to have the same attribute set, $\mathcal{A}ttr$, with possibly different tuples, i.e., $D_p[i]$ can differ from $D_s[i]$ for all i. n as used as the data size ($|D_s|$ or $|D_p|$, depending on the context).

Denial Constraints. Following previous work on related topics [10, 38, 53], we focus on Denial Constraints (DCs) on pairs of tuples. Given a database D over a schema $\mathcal{A}ttr$, a DC can be written as $\forall t_1, t_2 \in D, \neg(\phi_1 \wedge \ldots \wedge \phi_p)$, where ϕ_i is a predicate of the form $t_i[A_j] \circ v$ or $t_1[A_j] \circ t_2[A_j]$, for $i \in \{1, 2\}$, $A_j \in \mathcal{A}ttr$, $v \in Dom(A_j)$, and $\circ \in \{>, <, \geq, \leq, =, \neq\}$. DCs are a general form of integrity constraint, capturing multiple classes such as Functional Dependencies (FDs), Conditional FDs [4], and metric FDs [30]. If all tuple pairs in D satisfy a set of DCs Σ, we write $D \models \Sigma$.

Database repairs. Given a private database D_p and a synthetic database D_s generated from D_p over a schema $\mathcal{A}ttr$, D_r is a subset of D_s if $tids(D_r) \subseteq tids(D_s)$ and for all $i \in tids(D_r)$, $D_r[i] = D_s[i]$. We denote this by $D_r \subseteq D_s$. We consider repairs via tuple deletions, as in previous work [25, 34, 36]. Now, let Σ be a set of DCs, a *consistent subset* of D_s is a database $D_r \subseteq D_s$ such that $D_r \models \Sigma$.

Example 2.1. *Consider the databases D_p^E and D_s^E in Figure 1 based on an income dataset [9, 21, 31]. The database schema is $\mathcal{A}ttr = \{ID, Edu-num, Edu., Occu., Income, Age, Race\}$ and it includes $n = 4$ tuples. The probability of a tuple having the value $Undergrad$ is $\Pr_{D_p^E}(Edu. = Undergrad) = 0.5$ as two out of the four tuples contain this value. Now consider the following two DCs:*

(1) $\sigma_1 : \forall t_1, t_2 \neg(t_1[Edu] = t_2[Edu] \wedge t_1[Edu-num] \neq t_2[Edu-num])$
(2) $\sigma_2 : \forall t_1, t_2 \neg(t_1[Occu] = \text{'Prof'} \wedge t_2[Occu] = \text{'Craft-repair'} \wedge t_1[Income] < t_2[Income])$

Notice that $D_p^E \models \sigma_1, \sigma_2$ while $D_s^E \not\models \sigma_1, \sigma_2$. Specifically, (t_1, t_5), (t_1, t_6), (t_3, t_5), (t_3, t_6), and (t_5, t_6) violate σ_1, and (t_2, t_5), (t_5, t_6) violate σ_2. A repair of D_s^E would be removing t_5 and t_6.

2.2 Differential Privacy

Definition 2.2 (Neighboring databases). *Two databases D and D' are considered neighboring databases if D and D' differ by at most one tuple. We denote this relationship by $D' \approx D$.*

Definition 2.3 (Differential Privacy [17]). *An algorithm $\mathcal{A}$ is said to satisfy (ϵ, δ)-DP if for all $S \subseteq Range(\mathcal{A})$ and for all $D \approx D'$,*

$$\Pr[\mathcal{A}(D) \in S] \leq e^{\epsilon} \Pr[\mathcal{A}(D') \in S] + \delta.$$

The privacy cost is measured by the parameters (ϵ, δ), often called the *privacy budget*. The smaller ϵ and δ are, the stronger the privacy is.

For a function or algorithm required to satisfy DP, an important measure is the supremum over all pairs of neighboring databases, called the *sensitivity* of the function.

Definition 2.4 (Global sensitivity [18]). *Given a function $f : \mathcal{D} \to \mathbb{R}$, the sensitivity of f is defined as follows.*

$$\Delta_f = \max_{D' \approx D} |f(D) - f(D')|. \tag{1}$$

We consider zCDP [5] in this work rather than standard DP. We use zCDP for the ability to yield tight bounds on privacy loss for multiple differentially private data releases, and for its interpretability since it only has a single privacy parameter, which corresponds directly to the privacy loss.

Definition 2.5 (Zero-Concentrated Differential Privacy (zCDP) [5]). *A mechanism $\mathcal{A}$ is said to be ρ-*zero-concentrated differential private*, or ρ-zCDP for short, if for any neighboring datasets D and D' and all $\alpha \in (1, \infty)$ it holds that*

$$D_\alpha(\mathcal{A}(D) \| \mathcal{A}(D')) \leq \rho\alpha$$

where $D_\alpha(\mathcal{A}(D) \| \mathcal{A}(D'))$ denotes the Rényi divergence of the distribution $\mathcal{A}(D)$ from the distribution $\mathcal{A}(D')$ at order α [48].

This ensures that no single individual contributes too much to the final output of the randomized algorithm by bounding the difference of the distributions when computed on neighboring databases.

All mechanisms that satisfy zCDP also satisfy classic DP [16]. Given the privacy parameter ρ the zCDP guarantee can be translated into a classical DP guarantee as follows.

Theorem 2.6 (zCDP to DP [5]). *The following relationship holds:*

(1) If a mechanism $\mathcal{A}$ satisfies ρ-zCDP it satisfies $(\rho + 2\sqrt{\rho \log \frac{1}{\delta}}, \delta)$-DP for any $\delta > 0$.
(2) If a mechanism $\mathcal{A}$ satisfies ϵ-DP it satisfies $\frac{1}{2}\epsilon^2$-zCDP.

DP also ensures several properties that allow it to be used in complex scenarios without the need for privacy budget increases.

Proposition 2.7 (Sequential composition for zCDP [5, 46]). *Let $\mathcal{A}_1 : \mathcal{D} \to \mathcal{Y}$ and $\mathcal{A}_2 : \mathcal{Y} \to \mathcal{Z}$. Suppose $\mathcal{A}_1$ satisfies ρ_1-zCDP, $\mathcal{A}_2(\cdot)$ satisfies ρ_2-zCDP. Define $\mathcal{A}_3 : \mathcal{D} \to \mathcal{Z}$ by $\mathcal{A}_3(D) = \mathcal{A}_2(\mathcal{A}_1(D))$. Then, $\mathcal{A}_3$ satisfies $(\rho_1 + \rho_2)$-zCDP.*

The Exponential Mechanism (EM). EM [44] is aimed at selecting the best candidate from a specified domain $\mathcal{R}$. Formally, given a utility function that takes a candidate from a set $r \in \mathcal{R}$ and a private dataset D_p, and outputs a real number, the EM outputs a candidate $r \in \mathcal{R}$ with probability proportional to $exp(\frac{\epsilon u(D_p, r)}{2\Delta_u})$. Intuitively, u assigns larger values to preferable candidates with respect to the database and this is reflected by larger probability values.

Definition 2.8 (The Exponential Mechanism (EM) [45]). *Given D, a set of candidates $\mathcal{R}$, a utility function $u : \mathcal{R} \times \mathcal{D} \to \mathbb{R}$, where $\mathcal{D}$ is a set of datasets, and a privacy parameter ϵ, the exponential mechanism that selects and outputs $r \in \mathcal{R}$ with probability proportional to $\exp\left(\frac{\epsilon \cdot u(D,r)}{2\Delta_q}\right)$ is ϵ-DP.*

In particular, EM also satisfies zCDP by using Theorem 2.6.

3 SYNTHETIC DATA REPAIR

We now provide the required definitions for our framework and discuss the two problems related to synthetic data repair under DP.

We first formalize the distance measure for synthetic data repair and define our problem. We focus on two-way marginal, as these types of marginals have been proven to generate faithful synthetic data in prior work [40, 43, 64, 65], and define two-way marginal constraints (2-MCs). See Table 2 for notations.

Definition 3.1 (2-way marginal). *Given a pair of attributes $A_i, A_j \in \mathcal{A}ttr$, and a non-empty dataset D over $\mathcal{A}ttr$, the 2-way marginal of $a_i \in Dom(A_i), a_j \in Dom(A_j)$ in D is a normalized count $\sum_{t \in D} \mathbb{1}[t[A_i] = a_i \wedge t[A_j] = a_j]/n$ and is denoted by $m_D(i, j)$.*

We now define a 2-way marginal constraint or 2-MC, drawing on the representation constraints defined in [34], which can be viewed as 1-way marginal constraints. Intuitively, 2-MCs capture important and relevant statistical properties that we wish to maintain.

Definition 3.2 (2-way marginal constraint (2-MC)). *Let $\mathcal{A}ttr$ be a schema and $A_1, A_2 \in \mathcal{A}ttr$ a pair of attributes with two values $(a_1, a_2) \in Dom(A_1) \times Dom(A_2)$. A 2-way marginal constraint (2-MC) is $m_D(A_1 = a_1, A_2 = a_2) \geq \tau$, where $\tau \in [0, 1]$.*

If $\tau = 0$ the constraint can be ignored. For a pair of attributes A_1, A_2 where the lower-bound constraints on their marginals sum up to a 1 (formally, if we have l lower-bound constraints $m^i(A_1 = a_1, A_2 = a_2) \geq \tau_i$ for all $a_1, a_2 \in Dom(A_1) \times Dom(A_2)$ where $|Dom(A_1) \times Dom(A_2)| = k$, and $\sum_i^l \tau_i = 1$), the constraint has to exactly satisfy the proportions, i.e., $m^i(A_1 = a_1, A_2 = a_2) = \tau_i$ for all i. Otherwise, if the sum of the thresholds τ_i for all the 2-MCs over the attributes A_1, A_2 is larger than 1, the constraint is infeasible.

For a more compact notation (see Table 2), we use $m_D(i, j, \tau)$ instead of $m_D(A_i = a_i, A_j = a_j) \geq \tau$ and use the same notation as for DCs to indicate that a database satisfies a 2-MC: $D \models m_D(i, j, \tau)$. We denote the set of marginal constraints by $\mathcal{M}$.

We now define a synthetic data repair and an optimal synthetic data repair, drawing inspiration from work on data repair [34, 36].

Definition 3.3 (Synthetic data repair). *Given a database D, a set of 2-MCs $\mathcal{M}$, and a set of DCs Σ, a synthetic data repair of D is a consistent subset $D_r \subseteq D$ such that $D_r \models \Sigma$ and $D_r \models \mathcal{M}$. An* optimal *synthetic repair $D_r \subseteq D_s$ is a repair such that for any other repair $D'_r \subseteq D_s$, $|D_r| \geq |D'_r|$.*

We derive Definition 3.3 from the classic data repair problem, where the optimal repair minimizes the number of deletions while ensuring the satisfaction of the DCs and, in our case, the 2-MCs, which is adapted from prior work [34]. Specifically, we extend [34] to support 2-way marginals instead of 1-way marginals. By minimizing the number of deletions in the desired repair D_r, we avoid scenarios in which we are left with a small number of tuples whose marginals are closest to the marginals in D_p. Note that the set of provided 2-MCs $\mathcal{M}$ does not need to contain all 2-way marginals of the private data.

Example 3.4. *Reconsider the databases D_p^E and D_s^E in Figure 1 and the 2 DCs from Example 2.1. An example of 2-MCs that hold in D_p^E are $m_{D_p^E}(Inc = 50K, Edu = Under) \geq 0.5$ (violated by D_s^E) and $m_{D_p^E}(Edu = Under, Race = AA) \geq 0.25$, where $Under$ is short for $Undergrad$. To get an optimal synthetic data repair of D_s^E we can remove t_5 and t_6. In Example 2.1 we showed that this repair satisfies both DCs, and we can observe that it further satisfies both 2-MCs.*

Problem definitions. We now define the problems addressed in this paper to achieve a framework for synthetic data repair. The first problem focuses on obtaining 2-MCs from the private data that will guide the repair process towards a database that preserves specific important statistics. It is motivated by previous work [34] that showed the importance of explicitly including key representative statistics in the repair process.

Problem 3.5 (Private 2-MC generation). *Given a synthetic database D_s generated from a private database D_p, and a privacy parameter ρ, compute a set of 2-MCs $\mathcal{M}$ while satisfying ρ-zCDP.*

The second part of the problem focuses on repairing the synthetic data in light of the obtained 2-MCs and the DCs.

Problem 3.6 (Synthetic data repair). *Given a synthetic database D_s, a set of 2-MCs $\mathcal{M}$, and a set of DCs Σ, find an optimal synthetic data repair for D_s.*

We note that some data repair approaches do attempt to preserve the statistics that exist in the data [53]. However, in our problem setting, we aim at *satisfying specific statistical constraints and these may even differ from the current database statistics*, as we emphasize faithfulness to the private data, rather than to the synthetic data. Indeed, recall Example 1.1 where the synthetic data generated by various systems had a significant number of 2-MCs violations. Thus, we cannot rely on 2-MCs from the synthetic data, and resort to privately obtain them from the original data.

We then aim to provide a repair with a small distance from the initial synthetic database. However, finding the minimum synthetic data repair is NP-hard, as we now show.

Hardness of optimal synthetic data repair. The capture of statistics as constraints that can be viewed as 1-way marginals on the repaired data has recently been studied in a different context by Liu et al. [34]. Problem 3.6 can therefore be viewed as a generalization of that work as we consider adherence to more general constraints. From this work, we immediately inherit that the problem is hard when $|\Sigma| = 1$. However, since 2-MCs are more general, we get the following additional insight. This result assumes that the schema is constant, while the dataset and 2-MCs are inputs.

Proposition 3.7 (Hardness of Problem 3.6). *The decision version of Problem 3.6 is NP-hard even when $\Sigma = \emptyset$.*

We defer all proofs to [7] due to space limitations.

4 COMPUTING SYNTHETIC DATA REPAIR

We next present two approaches for solving Problem 3.6: (1) an ILP formulation of the problem that yields an exact solution, and (2) a heuristic approach relying on a reduction to a weighted vertex cover problem with dynamic weights.

4.1 An ILP Formulation

We next detail an ILP formulation of the problem. Each variable x_i represents a tuple t_i in the relation D. A tuple t_i remains in the relation after the repair only if $x_i = 1$ in the solution to the program. The goal of the ILP is to maximize the number of remaining tuples while ensuring that all conflicts are resolved, through the constraint $x_i + x_j \leq 1$ for every pair of conflicting tuples t_i, t_j, and ensuring that all marginal constraints are satisfied through the constraint $\sum_{l:t_l[A_i]=a_1,t_l[A_j]=a_2} x_l \geq \tau \cdot \sum_l x_l$. The correctness of the program is established by the following proposition (proof in [7]).

Proposition 4.1. *Given a synthetic database D_s, a set of DCs Σ, and a set of 2-MCs $\mathcal{M}$, a solution to the ILP in Equation* (2) *is an optimal synthetic data repair.*

$$
\begin{aligned}
&\textbf{Maximize} \sum_{i\in[1,n]} x_i \quad \textbf{subject to:} && (2)\\
&x_i + x_j \leq 1, \quad \text{for all } t_i, t_j \text{ conflicting on a constraint in } \Sigma\\
&\sum_{l:t_l[A_i]=a_1,t_l[A_j]=a_2} x_l \geq \tau \cdot \sum_{l\in[1,n]} x_l, \ \forall\, m_{D_s}(i,j,\tau) \in \mathcal{M}\\
&x_i \in \{0,1\} \quad \forall\, i \in [1,n]
\end{aligned}
$$

Example 4.2. *Consider the databases in Figure 1b, the two DCs from Example 2.1, and the two 2-MCs from Example 3.4. The corresponding ILP contains the DC-expressing constraints $x_1 + x_5 \leq 1$, $x_1 + x_6 \leq 1$, $x_2 + x_5 \leq 1$, $x_3 + x_5 \leq 1$, $x_3 + x_6 \leq 1$, $x_5 + x_6 \leq 1$, and the 2-MC-expressing constraints $\sum_{l:t_l[Inc]=50K,t_l[Edu]=Under} x_l \geq 0.5 \cdot \sum_{l\in[1,6]} x_l$, and $\sum_{l:t_l[Edu]=Under,t_l[Race]=AA} x_l \geq 0.25 \cdot \sum_{l\in[1,6]} x_l$.*

Note that the size of the ILP can be large as the number of 2-MCs is $|\mathcal{M}| \leq \binom{|\mathcal{A}ttr|}{2} \cdot max_{A\in\mathcal{A}ttr}|Dom(A)|^2$ and the number of conflicts in Σ is bounded by $O(|D|^2)$, making the bound on the program size $O(|D|^2 + \binom{|\mathcal{A}ttr|}{2} \cdot max_{A\in\mathcal{A}ttr}|Dom(A)|^2)$.

Equation (2) requires that a repair satisfies *both* the DCs and the 2-MCs and thus works even when the synthetic data satisfies the DCs and only violates the 2-MCs. As a result and in line with previous work [34], we show that the main obstacle in its use is not necessarily its runtime (despite the NP-hardness result, modern solvers are quite efficient), but that it often leads to the deletion of a substantial part of the database and thus significantly increases the distance between the repaired data and the original synthetic one (see Section 6). This motivates us to explore a heuristic solution.

4.2 A Weighted Vertex Cover Solution

In classic data repair scenarios, we only contend with integrity constraints. This allows the problem of finding the maximum repair to be naturally translated to a vertex cover (VC) problem by modeling each tuple as a vertex and each conflict as an edge. Formally, given a database D_s with a set of constraints Σ, the *conflict graph* [10, 37] $G = (V, E)$ is defined by $v_i \in V$ for each $t_i \in D_s$, and $\{v_i, v_j\} \in E$ if there is a constraint $\sigma \in \Sigma$ such that $\{t_i, t_j\} \not\models \sigma$. Hence, a minimum VC, C, maps exactly to the set of vertices that must be removed from D_s to obtain the maximum repair [38]. Since Definition 3.3 involves *marginal constraints as well as DCs*, we need a formulation that considers both. Here, we can turn to the *weighted* VC problem. The input is a graph with a cost function for each vertex $G = (V, E, w)$, where $w : V \to \mathbb{R}^+$, and outputs the VC (a set of vertices adjacent to all edges) with the lowest weight. Intuitively, w should map each vertex to the cost of its removal in terms of the effect it will have on violating the given marginal constraints. As opposed to Equation (2) in Section 4.1, this approach is DC violation-driven, i.e., it is only triggered when DC violations exist.

Designing a weight function. Intuitively, we aim to express the 2-MCs with w so that the minimum weighted VC will approximate the minimum synthetic data repair. To do so, we distill several desiderata that our weight function should satisfy to adequately represent the contribution of each vertex to the marginal constraints satisfaction. These desiderata are process-oriented to reflect the change in the database and the distance to a repaired database that will satisfy the 2-MCs as tuples are being selected for the cover (i.e., removed from the database). First, a rudimentary requirement for w is that it should be *dynamic*, as opposed to the classic setting in the weighted VC problem, to *reflect a process of repairing the data* and indicate the weight of a vertex based on the evolving state of the database at each step of the repair. Second, prior to weight normalization, we aim for $w(v)$ to be *non-negative* for all $v \in V$ and 0 iff removing v will cause each 2-MC to be satisfied in isolation. This requirement mirrors the positivity requirement for inconsistency measures [50]. In particular, for each vertex, we consider each 2-MC in isolation and we define $w(v)$ to be sum of the minimal number of tuples needed to satisfy each 2-MC after the hypothetical removal of v. We sum the sizes of the minimum required repair for each *single* 2-MC, which can be done in polynomial time, as opposed to computing the minimum repair size for a *set* of 2-MCs (shown to be NP-hard by Proposition 3.7).

Definition 4.3 (Dynamic weight function). *Let D_s be a synthetic dataset, $G = (V, E)$ be the conflict graph of D_s and $\mathcal{M}$ be a set of 2-MCs. The weight function of a vertex v_t is defined as:*

$$r_i(v_t) = \sum_{m_{D_s}(l,j,\tau)\in\mathcal{M}} min_{D_r \subseteq D_s \setminus \{t\}.\ D_r \models m_{D_r}(l,j,\tau)} |D_s \setminus D_r|$$

We normalize this function to obtain the final weight function:

$$w_i(v_t) = \frac{r_i(v_t) - r_{i,min}(v_{t'}) + \mu}{r_{i,max}(v_{t'}) - r_{i,min}(v_{t'}) + \mu}$$

where $r_{i,max}(v_{t'})$ ($r_{i,min}(v_{t'})$) is the maximum (minimum) r_i value at step i and μ is a small number meant to avoid division by zero.

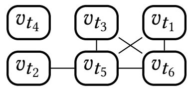
Intuitively, the larger the sum, the more 'expensive' it is to remove t from D_s and thus the higher the weight of v_t in the graph.

Additionally, we normalize the degrees of the vertices in the conflict graph such that for each $v \in V$

$$ndeg_i(v) = \frac{deg_i(v) - min_{u\in V} deg_i(u) + \mu}{max_{u\in V} deg_i(u) - min_{u\in V} deg_i(u) + \mu} \quad (3)$$

The normalization formula of the weights and degrees aims to place all weights in $[0, 1]$ and, moreover, ensures that low weights are close to 0, thus making them easily discernible from high weights which will be closer to 1 (as opposed to just dividing by the maximum degree for example, which may lead to similar normalized degrees if the distribution is close to uniform). Vertices with degree 0 are ignored, as they do not contribute to the vertex cover.

Example 4.4. *The following graph is the conflict graph of D_s^E in Figure 1b based on the DCs in Example 2.1:*



The initial non-normalized weight of the tuple vertex t_5 is $r_1(v_{t_5}) = 1$ for the two 2-MCs defined in Example 3.4. It is derived as follows. If we remove t_5 from D_s^E, the data size will be 5 and the number of tuples related to each 2-MC is 2 which means the normalized count for each 2-MC is $2/5 = 0.4$. To satisfy the first 2-MC, the normalized count must be above 0.5, thus we need to remove a single tuple that is not related to the 2-MC (t_2, t_4 or t_6) so we will get $2/4 = 0.5$. The second 2-MC is already satisfied as it requires a relative size of 0.25, so no more tuples need to be removed. Thus, $r_1(v_{t_5}) = 1$.

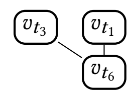
Algorithm 1: Data repair via dynamic weighted VC

input : Conflict graph $G = (V, E)$ based on D_s, the weight function w, and 2-MC set $\mathcal{M}$.
output : Vertex cover *Removed*.

```
1  Initialize Removed = ∅ and index i = 0;
2  while E ≠ ∅ do
       /* Dynamic weight updates.                         */
3      foreach v ∈ V do
4          w_i(v) ← RecomputeWeight(D_s, M);
5      v* ← argmin_{v∈cand} w_i(v) / ndeg_i(v) ;
6      D_s ← D_s \ {t_{v*}};
7      Removed.append(t_{v*}) ;
8      Delete all edges of v* and the resulting isolated vertices
         from G, denote the new set of vertices by V and the
         new set of edges as E;
9      i ← i + 1;
10 return Removed;
```

A weighted vertex cover algorithm. We now harness the weight function in Definition 4.3 to obtain a VC-based algorithm for synthetic data repair. Algorithm 1 shows the pseudo code. Our algorithm is inspired by the 2-approximation algorithm for weighted vertex cover with *fixed* weighted developed by Clarkson [11]. To handle dynamic weights, we add a weight-update step in the algorithm as follows. The algorithm initializes the sequence of the tuples to be removed in Line 1 and iterates in a While loop while edges remain in the conflict graph (Line 2). In the inner loop in Line 3, the weights of all vertices are updated and assigned to them (Line 4). The vertex that minimizes the ratio between its weight and its normalized degree is then chosen for removal in Line 5, its corresponding tuple is removed in Line 6, and the vertex is appended to the sequence of removed vertices in Line 7. Intuitively, the vertex that we *remove* is the vertex that should have the lowest comparative weight (its removal is least expensive in terms of marginals) and the highest relative degree (its removal is most beneficial in terms of a cover). In Lines 8 and 9 the graph is updated by removing edges that were adjacent to the removed vertex, and then the number of steps is increased. When no more edges exist in the graph, the resulting sequence is returned in Line 10.

Example 4.5. *We illustrate Algorithm 1 through D_s^E depicted in Figure 1b whose conflict graph is presented in Example 4.4 and using the 2-MCs from Example 3.4. After initialization in Line 1, the algorithm begins its first iteration by computing the weights in Line 3, Line 4. For example, the calculation of $w_1(v_{t_5})$ is illustrated in Example 4.4. The weights would be $r_1(v_{t_1}) = r_1(v_{t_3}) = 3, r_1(v_{t_2}) = r_1(v_{t_5}) = r_1(v_{t_6}) = 1$. After finding all the weights, we can derive the minimal and maximal weight, and normalize with $\mu = 10^{-7}$ according to Definition 4.3, as done in lines 3, 4. We get $w_1(v_{t_1}) = w_1(v_{t_3}) = 1, w_1(v_{t_2}) = w_1(v_{t_5}) = w_1(v_{t_6}) = \frac{1}{2+10^{-7}} \cdot 10^{-7} \approx 0.5 \cdot 10^{-7}$. The maximal degree is the degree of v_{t_5} which is 4 and the minimal degree is that of v_{t_2} which is 1 (ignoring v_{t_4} which is isolated). Using $\mu = 10^{-7}$ we can find the normalized degree of each vertex according to Equation (3). By dividing the weights with the normalized degrees (Line 5), we get that the ratios are $\frac{3+10^{-7}}{1+10^{-7}} \approx 3$ for v_{t_1} and v_{t_3}, $\frac{3+10^{-7}}{2+10^{-7}} \approx 1.5$ for v_{t_2}, $\frac{10^{-7}}{2+10^{-7}} \approx 0.5 \cdot 10^{-7}$ for v_{t_5} and $\frac{10^{-7}(3+10^{-7})}{(2+10^{-7})^2} \approx 0.75 \cdot 10^{-7}$ for v_{t_6}. Since v_{t_5} is the vertex minimizing the ratio, we choose it in Line 5, remove it from the data (Line 6), and update the conflict graph (Line 8). This is the updated conflict graph:*



In the second iteration of Line 2, the weight of the remaining tuples are updated in lines 3, 4 to $r(v_{t_1}) = r(v_{t_3}) = 2, r(v_{t_6}) = 0$. Repeating the same process as before would lead to the removal of t_6 as it minimizes the ratio (Line 5). After this iteration, there are no edges left in the graph, so the algorithm halts and returns the cover $\{t_5, t_6\}$. Removing these tuples returns a dataset that satisfies all the DCs and the 2-MCs.

Complexity analysis. The complexity of Algorithm 1 is $O(|\mathcal{M}|n^2)$. The algorithm operates in a nested loop fashion, where the While loop in Line 2 iterates until no edges remain in the graph and the inner 'For' loop in Line 3 iterates over all vertices in the current graph. The weight update step can be done efficiently in $O(|\mathcal{M}|)$ for each vertex by doing so incrementally (details in [7]), thus taking $O(|\mathcal{M}||V|)$ for all vertices. Since every iteration includes the removal of at least one vertex (Line 8), the complexity is $O(|\mathcal{M}||V|^2)$, where $|V| = n$.

5 CHOOSING MARGINAL CONSTRAINTS

In this section, we tackle Problem 3.5 by first giving a general algorithm that takes a utility function that ranks the marginals and outputs the top-k ones, while satisfying DP, and then describe a low sensitivity ranking function that may be plugged into it.

Obtaining marginals. To obtain the most effective marginals, we utilize a selection mechanism based on the exponential mechanism (Definition 2.8). We present a general approach to obtain the top-k marginals for a general utility function.

We limit the number of marginals to a specific amount as we aim to control the privacy loss incurred at this stage. Our method for selecting the top-k marginals, shown in Algorithm 2, builds on the approach of [43] and integrates the one-shot top-k mechanism [14, 15]. In DP applications, this mechanism is commonly employed to identify the top-k candidates rather than a single top element, which is the typical objective of the Exponential Mechanism (EM).

Algorithm 2: Obtain k representative noisy 2-MCs

input : A private and synthetic datasets D_p, D_s, a utility function u, a privacy parameter for top-k ρ_{Topk}, a number of desired 2-MCs k, and a privacy parameter for threshold computation ρ_{marg}.

output : Set of k representative 2-MCs $\mathcal{M}$.

1 Compute all 2-way marginals of D_p and of D_s, $m_D(i,j)$;

2 Let $\sigma = 2\Delta_u\sqrt{\frac{k}{8\rho_{Topk}}}$, $\mathcal{M} = \emptyset$;

3 **foreach** *marginal* $m_{D_p}(i,j)$ **do**

4 $\quad score_u(m_{D_p}(i,j)) = u(m_{D_p}(i,j)) + Gumbel(\sigma)$;

5 **end**

6 Sort the marginals based on $score_u$ and take the top-k;

7 Let $\mathcal{T}_k$ be the top-k marginals by $score_u$;

```
/* Compute the noisy 2-MCs threshold of the top-k
   marginals using ResidualPlanner [62].            */
```

8 $\mathcal{M} = ResidualPlanner(D_p, \mathcal{T}_k, \rho_{marg})$;

9 **return** $\mathcal{M}$;

In Line 1, the algorithm computes the 2-way marginals $m_{D_p}(i,j)$ and $m_{D_s}(i,j)$. It sets the privacy budget value based on k and the privacy parameter ρ_{marg} in Line 2. For each marginal, the algorithm then computes its utility score via the function u and perturbs it by adding Gumbel noise with scale $\sigma = 2\Delta_u\sqrt{k/(8\rho_{Topk})}$, where Δ_u denotes the sensitivity of the utility function (Lines 3 and 4). The marginals are then sorted in decreasing order according to these noisy scores in Line 6, and the first k are taken to the set $\mathcal{T}_k$ in Line 7. The resulting procedure satisfies ρ_{Topk}-zCDP [6, 13–15, 54], as it is equivalent to sequentially executing k instances of the exponential mechanism [19]. For the marginals in $\mathcal{T}_k$, the algorithm computes the noisy 2-MCs using ResidualPlanner [43] (the threshold τ is noisy) with a privacy parameter ρ_{marg} in Line 8 and adds them to $\mathcal{M}$. Finally, it returns them in Line 9.

While the number of 2-way marginals can be extremely large, the privacy budget for the selection step (Line 3–Line 7) is only $\sqrt{8\rho_{Topk}/k}$, i.e., it avoids a linear blowup in the size of the marginal set due to the nature of the One-Shot Top-k mechanism. We thus get the following result, slightly abusing notation to denote by $\mathcal{M}_0$ the set of all marginals (proof in [7]).

Proposition 5.1 (Algorithm 2 guarantees). *Given the utility function u with sensitivity Δ_u, a set of marginals $\mathcal{M}_0$, a privacy parameter ρ_{Topk}, and a natural number k, the following holds:*

(1) *Algorithm 2 satisfies $(\rho_{marg} + \rho_{Topk})$-zCDP.*

(2) *Denote by $OPT^{(i)}$ the i-th highest (true) score, and by $\mathcal{M}_0^{(i)}$ the i-th noisy private marginal selected by the One-shot Top-k mechanism. For $\forall t$ and $\forall i \in \{1, 2, \ldots, k\}$, we have*

$$Pr[u(\mathcal{M}_0^{(i)}) \le OPT^{(i)} - \frac{2\Delta_u}{\sqrt{8\rho_{Topk}/k}}(\ln(|\mathcal{M}_0|) + t)] \le e^{-t}$$

Utility function and attribute focus. We next describe the utility function used to evaluate the marginals. Although we focus on a specific function here, Algorithm 2 and its guarantees are general and can take any utility function as input. The function we use here quantifies the utility of a 2-way marginal of D_s as its distance from the corresponding 2-way marginal of D_p.

Definition 5.2 (2-way marginals utility). *Given a private database D_p, a synthetic database D_s over $\mathcal{A}ttr$, and two attributes $A_i, A_j \in \mathcal{A}ttr$, the utility function of $m_{D_p}(i,j)$ is $u(m_{D_p}(i,j)) = \|m_{D_p}(i,j) - m_{D_s}(i,j)\|$.*

Intuitively, this function aims to prioritize the most poorly estimated marginals is the synthetic data, making it less faithful to the private data. The sensitivity of the function is low, allowing us to compute it with relative accuracy under DP (proof in [7]).

Proposition 5.3. *The sensitivity of u is $\Delta_u = \frac{1}{n}$.*

When users have a preference for a specific subset of attributes, Algorithm 2 can consider only this subset and aim to get the least well-represented 2-MCs of these attributes alone (as in Example 1.1).

Example 5.4. *Consider the databases presented in Figure 1. Suppose that we are interested in the distribution of the Occupation and Race attributes, thus the 2-MCs we wish to extract are only for this pair. In D_p^E, each value combination of Occupation and Race is unique, thus the marginal of each combination is* 0.25. *In D_s^E, only the value combination (Prof., AA) appears more than once thus its marginal is* 0.33 *and the rest have marginal of* 0.167. *If a combination does not appear in a database, its marginal for that database is* 0. *We therefore get $u(Manager{,}AA) = u(Craft\text{-}repair, SA) = u(Farming{,}C) = u(Prof{,}AA) = 0.083$, and $u(Prof{,}AA) = 0.167$, making the marginal $(Prof., AA)$ the most disparate.*

6 EXPERIMENTS

We perform an experimental evaluation of our framework, aimed at answering the following questions: (1) How does our framework fare in terms of repair quality compared to existing baselines? (2) How does Algorithm 1 scale compared to existing baselines? (3) How do the privacy budget components affect our 2-MCs (Algorithm 2) and the quality of the repair?

Summary of our findings. The experiments reveal several tradeoffs. First, improving the satisfaction of specific 2-MCs generally degrades other faithfulness measures. The dynamic weighted VC approach balances these objectives, satisfying more 2-MCs than the baselines while limiting harm to other quality metrics. In contrast, the ILP approach in Equation (2) enforces all 2-MCs and significantly reduces other quality attributes and incurs a high deletion rate, whereas approaches that ignore 2-MCs better preserve more data and general statistics. A second tradeoff concerns runtime: Algorithm 1 with dynamic weights improves quality but incurs higher computational cost. Moreover, reducing 2-MC violations does not necessarily improve overall faithfulness, since 2-MCs capture only selected marginals. As for privacy parameters, we find that ρ_{marg} has substantially greater impact on repair quality than ρ_{Topk} in Algorithm 2, suggesting that allocating more privacy budget to ρ_{marg} is preferable. Across datasets, Algorithm 1 with dynamic weights reduces 2-MC violations by more than 20% compared to baselines that focus on minimizing deletions, while retaining at least 10% more tuples than Equation (2). Its faithfulness (measured via TVD) is on average only 0.08 higher than an optimal DC repair, indicating limited distortion, but it incurs roughly a 10-fold runtime slowdown.

6.1 Setup

We tested variations of Equation (2) and Algorithms 1 and 2 by running experiments on a 20 CPU machine and 500GB RAM. We used Python 3.11. We used Pandas and DuckDB for dataset interaction. We use Gurobi for solving the ILPs, and igraph for representing conflict graphs.[1] Each experiment was repeated 8 times.

Datasets. We used the following datasets to run the experiments:

- **IPUMS-CPS [21]:** A dataset of Current Population Survey from the U.S. Census Bureau with 3, 395, 604 tuples from the year 2022. The dataset contains 8 attributes: 6 categorical attributes and two numerical attributes. AGEP and WAGP which represent the age and income of a person which were bucketized.
- **Adult [31]:** The dataset is composed of 48, 842 tuples and 15 attributes and was derived from the US Census population survey. It contains demographic information similar to IPUMS-CPS.
- **Compas [51]:** This dataset was used by judges and parole officers for scoring a defendant's likelihood of re-offend (recidivism). It contains attributes such as race, sex, age, criminal past, etc. It contains 60,798 tuples and 17 attributes.
- **Tax [9]:** US personal tax record information including personal and demographic details with 999,910 tuples and 9 attributes.

If not stated otherwise, we limited the data size of the dataset to 10K tuples to ensure convergence. The constant in the weight function of Algorithm 1 (Definition 4.3) was set to $\mu = 10^{-10}$.

2-MCs. In order to replicate real usage of the framework, we employ Algorithm 2 with the entire set of attributes to generate noisy 2-MCs. If not stated otherwise, we generated $k = 100$ 2-MCs from the private data using the utility function in Definition 5.2. If not stated otherwise, we used privacy budgets $\rho_{Topk} = 1$ and $\rho_{marg} = 10$.

Denial Constraints. Most of the DCs used for the experiments are taken from previous work [34, 49, 53], where the authors manually discovered them. Several new DCs were found to hold due to bucketization (for the full list of DCs see [7]).

Data generation systems. In order to generate authentic synthetic data for the experiments, we used prominent private data generation algorithms. Those include a GAN-based approach [63] and marginal-based approaches [40, 41]. We do not use Kamino [23] since it repeatedly generated data with a significant amount of 2-MC violations (approximately 97%), and running our ILP to find the optimal repair consistently resulted in the deletion of the entire dataset. For hyperparameters, we use the vanilla version. All the generation systems use a privacy budget of $\rho = 11$, in order to match the default privacy budget of the 2-MCs generation algorithm.

Data preprocessing. To ensure the ability to convert the datasets to conflict graphs and use the data generation algorithms, we perform several basic steps to preprocess the data. Specifically, we bucketized numerical attributes and removed attributes with unique or close to unique values (such as id, phone numbers, etc.).

Noise injection. Despite synthetic data being inherently noisy, to control the amount of noise in the experiments, we use the Co-Noise procedure from prior work [35, 49] in Section 6.3. Co-Noise operates iteratively: in each iteration, it selects a DC and two tuples at random, then modifies the values of the attributes involved in that constraint so as to induce a violation.

[1] Source code: https://github.com/ItayChairmanHuji/PrivateSyntheticDataRepair

Our repair approaches. We examine three versions of the repair algorithms presented in this paper:

- **WVC-Vanilla:** Algorithm 1 with uniform constant, unchanging weights of 1 for all vertices.
- **WVC-Dynamic:** Algorithm 1 with the dynamic weight function from Definition 4.3 based on the 2-MCs.
- **Optimal-M:** Our proposed ILP from Section 4.1 that considers both DCs and 2-MCs and formulates the problem as a program that is inputted to an ILP solver.

Baselines. Since this work is the first to consider synthetic data repair, we compare Algorithm 1 to previous work for classic data repair that only considers integrity constraints and mainly attempts to minimize the number of changes made to repair the data, as opposed to minimizing the marginal distance.

- **Optimal-DC-Baseline [38]:** The ILP in Section 4.1 without any 2-MCs, which results in an optimal repair algorithm for DCs.
- **VC-approx-Baseline [60]:** Using the classic reduction of the data repair problem to the vertex cover by defining the conflict graph where each tuple is represented by a vertex and an edge connects each pair of tuples that violate an FD. The implemented 2-approximation algorithm is the classic greedy approach.

Quality measures. We use several measures for the repaired data faithfulness and the overhead for repairing it. For data faithfulness, we apply the standard measures [52, 57] of similarity and for data repair, we employ the a variation of deletion overhead [34]:

- **Deletion ratio:** This is the ratio of deleted tuples during the repair compared to the original size of the synthetic data $1 - \frac{|D_r|}{|D_s|}$.
- **2-MCs violations:** This measurement presents the relative number of 2-MCs that were not satisfied by a dataset $\frac{\#(\text{Violated 2-MCs})}{\#(\text{2-MCs})}$
- **Pairwise attribute similarity:** We measure the TVD of the 2-way marginal vectors ('two-way TVD' for short) of the original and repaired synthetic data. We do not make a distinction between the marginals considered in the constraints and those that are not considered in this measurement.
- **ML accuracy:** Here we use the synthetic data to train several classifiers and use them to classify the original data. We report the accuracy $\frac{TP+TN}{TP+FP+FN+TN}$ for a model trained on each of the 10 generated and repaired datasets. For this, we trained a Multi-Layer Perceptron (MLP), Random Forest, and Logistic Regression models on the repaired synthetic data and the private data.

6.2 End-to-End Analysis with WVC-Dynamic

This section evaluates the full pipeline, from private data through synthetic generation and 2-MC generation, to repair with the WVC-Dynamic approach, since, as we show in later sections, this approach strikes a balance between faithfulness (measured via TVD and ML accuracy), 2-MCs satisfaction, and deletion ratio. Starting with a private dataset, we generate synthetic data, compute noisy 2-MCs via Algorithm 2, and repair using Algorithm 1. The results demonstrate tradeoffs under realistic conditions: WVC-Dynamic reduces 2-MC violations while minimally affecting downstream ML accuracy, yet significantly affecting two-way TVD.

Table 3: Comparison of the generated synthetic data and the repaired data using WVC-Dynamic for our end-to-end analysis. The comparison includes # of DC violations, ratio of 2-MC violation after the repair (repaired data using WVC-Dynamic/synthetic), where 1 means no change pre- and post-repair, runtime ratio, and two-way TVD increase post-repair. Green (red) cells indicate post-repair improvement (deterioration).

Dataset	DC violations				2-MC violation ratio after repair			Runtime ratio of repair compared to generation			Two-way TVD ratio after repair		
	AIM	MST	PATE CTGAN	Repair	AIM	MST	PATE CTGAN	AIM	MST	PATE CTGAN	AIM	MST	PATE CTGAN
Adult	1.00e+06	7.10e+05	9.54e+06	0	0.7153	0.7283	0.6740	0.0993	9.0214	1.6641	3.8554	2.7854	1.1394
Census	2.58e+05	6.08e+04	1.92e+06	0	0.8605	0.9623	0.7952	0.0626	0.4960	1.2986	2.4412	1.2951	0.9650
Compas	2.91e+05	1.80e+05	6.74e+06	0	0.6311	0.7088	0.7440	0.0299	3.0882	3.7171	7.2499	1.9620	0.9255
Tax	3.45e+05	7.05e+05	3.15e+05	0	0.9563	1.0490	0.9972	0.4138	12.0344	0.3662	3.6030	3.6375	1.0231

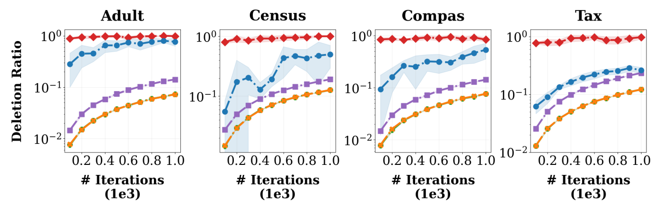
The left side of Table 3 reports DC violations in outputs of several synthetic generators. All exhibit substantial violations, including state-of-the-art models such as AIM that outputs data with up to 10^6 violations, motivating post-generation repair. The table (center left) also shows the ratio of 2-MC violations (synthetic vs. repaired). Since target 2-MCs are noisy for DP, even the private data does not fully satisfy them; moreover, synthetic data and 2-MCs are produced by separate noisy processes. Nonetheless, repaired data consistently reduces 2-MC violations relative to synthetic data, confirming that the repair integrates marginal constraints. The same table (center right) reports the runtime ratio between generation and repair. With AIM, generation dominates repair time which takes up to 13,000 seconds, motivating post-hoc repair rather than modifying the generator. With PATE-CTGAN, runtime is dataset dependent and repair can exceed generation time. This reflects a tradeoff: although PATE-CTGAN may generate data faster than AIM, it produces substantially more DC and 2-MC violations, increasing repair runtime cost (see Figure 5a).

Table 4: ML accuracy ratio (trained over the repaired data using WVC-Dynamic/the original synthetic data) across synthesizers. Values larger (smaller) than 1 indicate post-repair improvement (deterioration).

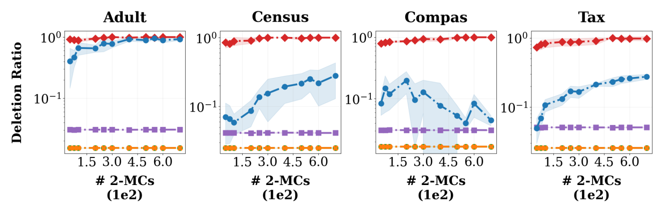
Dataset	AIM			MST			PATE-CT		
	LR	RF	MLP	LR	RF	MLP	LR	RF	MLP
Adult	0.5800	0.5157	0.6912	0.4463	0.4355	0.6239	0.8104	0.8694	1.0268
Census	0.9997	0.9953	0.9922	1.0001	0.9983	0.9911	0.7042	0.9906	0.7545
Compas	0.9498	0.9509	0.9571	0.9746	0.9760	0.9730	1.7775	1.3013	1.0062
Tax	0.9927	0.9883	0.9895	0.9615	0.8323	0.9714	1.0105	0.9997	0.9596

Table 4 compares downstream ML accuracy (repaired vs. synthetic), with models trained on synthetic or repaired data and tested on private data. Performance remains comparable across datasets, and accuracy differences are small, indicating that repair preserves utility while enforcing constraints. Finally, Table 3 (right side) reports the ratio of two-way TVD (synthetic vs. repaired). Repair typically causes a TVD increase, consistent with Figure 4, since optimization targets specific 2-MCs rather than the full joint distribution. DP noise in the target 2-MCs may also shift statistics (see Figure 4 in the sequel). Data generated and repaired from PATE-CTGAN shows higher TVD than AIM; although AIM has higher generation cost, it yields more faithful repaired data. The deletion ratio for each synthesizer is consistent with our results in the sequel. The data generated by AIM and MST is more easily repaired and WVC-Dynamic removes 35% of the data on average, while PATE-CTGAN creates data with a significantly higher number of violations that is harder to repair, yielding a ratio that exceeds 78%.

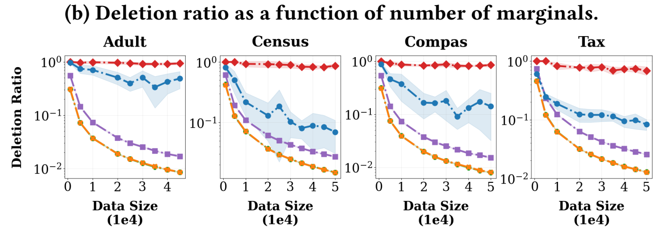
6.3 Repair Quality Analysis



(a) Deletion ratio as a function of number of Co-Noise iterations.



(b) Deletion ratio as a function of number of marginals.



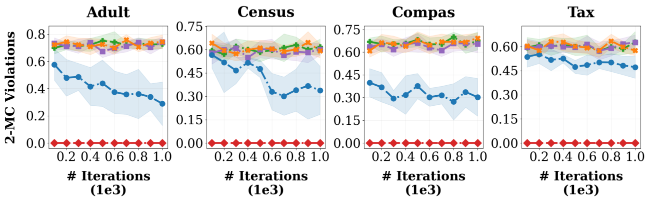
(c) Deletion ratio for each dataset as a function of data size.

+ Optimal-DC ◆ Optimal-M ■ VC-approx ● WVC-Dynamic ✱ WVC-Vanilla

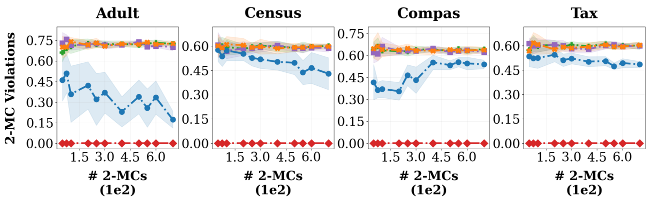
Figure 2: Deletion ratio as a function of different parameters.

Figures 2 to 4 evaluate the performance of our approaches against the baselines across three dimensions: data utility (deletion ratio), constraint satisfaction (2-MC violations), and statistical faithfulness (two-way TVD). Our results demonstrate that WVC-Dynamic effectively interpolates between the different approaches and baselines.

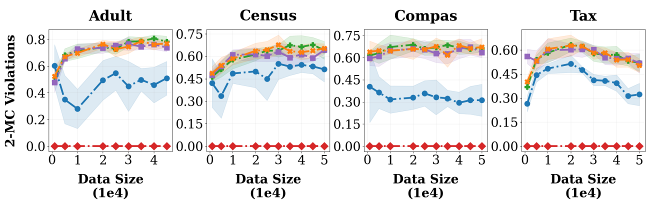
Deletion ratio. Figure 2 reports the deletion ratio, i.e., the fraction of tuples removed during repair with respect to different factors: Co-Noise iterations for noise injection, number of 2-MCs, and data sizes. Optimal-DC-Baseline and WVC-Vanilla completely overlap and provide a lower bound by minimizing cardinality loss to satisfy DCs; WVC-Vanilla closely tracks it. VC-approx-Baseline exhibits a higher deletion ratio, as it is an approximation of Optimal-DC-Baseline . Optimal-M forms an upper bound by enforcing all 2-MCs as hard constraints, leading to aggressive deletions. WVC-Dynamic balances both objectives, treating 2-MCs as



(a) 2-MC violations as a function of number of Co-Noise iterations.



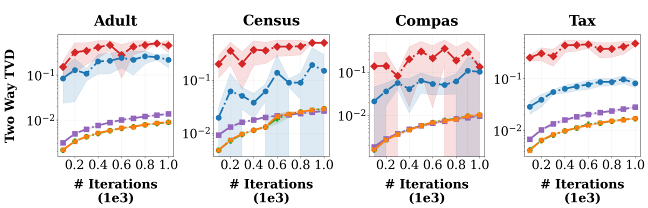
(b) 2-MC violations as a function of number of marginals.



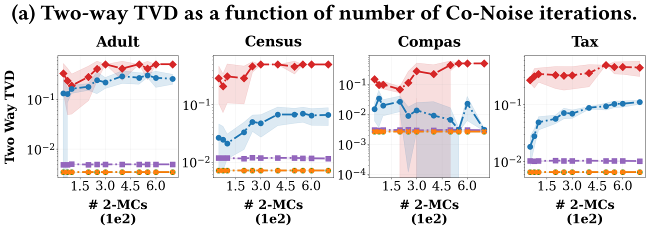
(c) 2-MC violations for each dataset as a function of data size.

+ Optimal-DC ◆ Optimal-M ■ VC-approx ● WVC-Dynamic × WVC-Vanilla

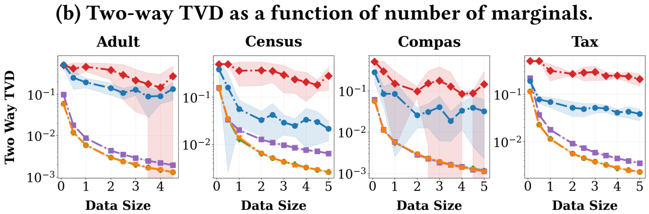
Figure 3: 2-MC violations as a function of different parameters.



(a) Two-way TVD as a function of number of Co-Noise iterations.



(b) Two-way TVD as a function of number of marginals.



(c) Two-way TVD for each dataset as a function of data size.

+ Optimal-DC ◆ Optimal-M ■ VC-approx ● WVC-Dynamic × WVC-Vanilla

Figure 4: Two-way TVD as a function of different parameters.

soft constraints and achieving substantially fewer deletions than Optimal-M while still accounting for statistical properties.

Figure 2a shows the deletion ratio versus Co-Noise iterations. Optimal-DC-Baseline, WVC-Vanilla, and VC-approx-Baseline increase roughly linearly (note that the graphs are log scaled) as more DC violations are introduced. Optimal-M slightly increases due to the introduction of more DC violations, which makes the problem harder, leading to more necessary deletions. The trend for WVC-Dynamic rises as well, remaining between the baselines since it considers 2-MCs without enforcing them strictly. For example, after 600 Co-Noise iterations on Compas, Optimal-DC-Baseline and WVC-Vanilla delete around 4.6% of the data, VC-approx-Baseline around 8.7%, Optimal-M around 90%, and WVC-Dynamic about 31%. Figure 2b varies the number of queried 2-MCs. Optimal-M increases steadily as more constraints shrink the ILP feasible region. Optimal-DC-Baseline, WVC-Vanilla, and VC-approx-Baseline remain constant. WVC-Dynamic generally increases but not monotonically, as a fixed DP budget implies more noise per marginal when more 2-MCs are queried, occasionally yielding looser or tighter constraints. For Tax with 600 2-MCs, Optimal-DC-Baseline and WVC-Vanilla delete around 2.5%, VC-approx-Baseline around 5%, Optimal-M deletes 98% and WVC-Dynamic about 26%. Figure 2c focuses on dataset size. Deletion ratios decrease for all methods as size grows, since repair cost becomes relatively smaller and the fixed Co-Noise magnitude yields a lower dirty-to-clean ratio. For Census at 10K tuples, WVC-Vanilla and Optimal-DC-Baseline delete about 7%, VC-approx-Baseline around 11%, Optimal-M about 93%, and WVC-Dynamic about 22%.

2-MC violations. Figure 3 reports the fraction of violated 2-MCs after repair. As expected, Optimal-M achieves zero violations. Optimal-DC-Baseline, VC-approx-Baseline, and WVC-Vanilla, which ignore 2-MCs, exhibit the highest violation rates and serve as worst-case baselines. WVC-Dynamic consistently reduces violations relative to them, showing effective integration of statistical targets. Figure 3a plots violations versus Co-Noise iterations. Optimal-DC-Baseline, WVC-Vanilla, and VC-approx-Baseline remain roughly constant, as minimal DC-only repairs lack the flexibility to improve marginal satisfaction. In contrast, WVC-Dynamic decreases 2-MC violations as noise increases: higher noise induces more DC repairs (Figure 2a), leading to more deletions and greater flexibility to remove tuples conflicting with 2-MCs. For example, after 400 Co-Noise iterations on Adult, WVC-Vanilla, Optimal-DC-Baseline, and VC-approx-Baseline violate about 90% of the 2-MCs, Optimal-M violates none, and WVC-Dynamic about 40%.

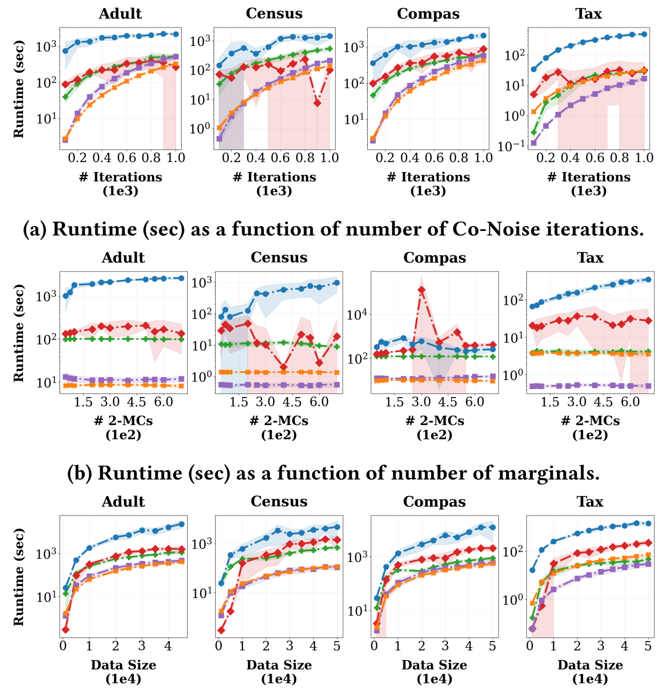
Figure 3b varies the number of 2-MCs. WVC-Vanilla, Optimal-DC-Baseline, and VC-approx-Baseline remain constant. WVC-Dynamic is dataset dependent: when distributions align with constraints, more 2-MCs reduce violations; when adversarial, additional targets introduce conflicts and increase violations, reflecting dependence on data topology. For 400 Co-Noise iterations on Adult, Optimal-DC-Baseline, WVC-Vanilla, and VC-approx-Baseline violate about 90% of the MCs, while WVC-Dynamic violates about 40%. With 300 2-MCs on Census, WVC-Vanilla, Optimal-DC-Baseline, and VC-approx-Baseline violate roughly 60%, and WVC-Dynamic about 50%. Finally, Figure 3c shows violation rates versus dataset size. Although larger datasets stabilize marginals, Figure 2c indicates lower deletion ratios, limiting distributional shifts. Thus violations may persist when 2-MCs are intrinsically hard to satisfy. For Tax at size 50K, Optimal-DC-Baseline, VC-approx-Baseline, and WVC-Vanilla violate about 50%, Optimal-M none, and WVC-Dynamic approximately 30% (recall from the previous paragraph that WVC-Dynamic balances the deletion ratio).

Statistical faithfulness of the repair. Figure 4 reports TVD between repaired and private datasets over all 2-way marginals, measuring global fidelity rather than individual 2-MC satisfaction. TVD is computed against the clean private data, reflecting reconstruction from noisy DP marginals. Across settings, WVC-Dynamic and VC-approx-Baseline lie between the extremes: Optimal-M has the highest TVD, while WVC-Vanilla and Optimal-DC-Baseline have the lowest (reflecting the opposite trend of deletion ratio in Figure 2). This stems from three factors. First, Optimal-M strictly enforces noisy 2-MCs and overfits DP noise, whereas WVC-Vanilla, Optimal-DC-Baseline, and VC-approx-Baseline ignore them and remain closer to the private distribution by minimizing deletions. Second, repairs optimize only a subset of 2-MCs, while TVD aggregates over all marginals, so local enforcement may distort global statistics. Third, 2-MCs are inequality thresholds, so satisfying them can overshoot private marginals and increase TVD. Figure 4a shows TVD increasing with noise intensity. With 500 Co-Noise iterations on Compas, Optimal-DC-Baseline, WVC-Vanilla, and VC-approx-Baseline are near 0.006 TVD, Optimal-M is about 0.3, and WVC-Dynamic about 0.06. Figure 4b shows TVD does not decrease with more 2-MCs, since under a fixed privacy budget additional marginals add more DP noise, leading Optimal-M and WVC-Dynamic to optimize inaccurate targets. For Tax with 600 2-MCs, WVC-Vanilla, Optimal-DC-Baseline are near 0.007, VC-approx-Baseline is around 0.01, Optimal-M about 0.46, and WVC-Dynamic about 0.1 TVD. Finally, Figure 4c shows TVD decreasing with dataset size due to asymptotic stability. WVC-Dynamic tracks the minimal-repair approaches while satisfying more 2-MCs than Optimal-DC-Baseline, VC-approx-Baseline, and WVC-Vanilla (see Figure 3c); for Adult at 40K, the two-way TVDs of WVC-Vanilla, Optimal-DC-Baseline are around 0.0015, where VC-approx-Baseline is approximately 0.002, Optimal-M is around 0.15 and WVC-Dynamic is 0.09.

6.4 Scalability Analysis

Figure 5 reports the runtime in seconds, excluding preprocessing such as conflict graph construction. Overall, WVC-Dynamic is slower than both ILP approaches, WVC-Vanilla and VC-approx-Baseline. It iteratively sweeps the conflict graph until all edges are resolved; although each sweep is cheap, iterations accumulate overhead in dense graphs. In contrast, Optimal-M and Optimal-DC-Baseline leverage Gurobi's presolve and pruning, which often solve these NP-hard formulations faster in practice.

Figure 5a shows runtime increasing with Co-Noise iterations as violations grow and conflict graphs densify. VC-approx-Baseline and WVC-Vanilla are usually the fastest due to greedy traversal. Optimal-M and Optimal-DC-Baseline fluctuate; Optimal-M is usually slower due to marginal constraints, but under high noise infeasibility may be detected early, occasionally terminating faster than Optimal-DC-Baseline. For 800 Co-Noise iterations



(a) Runtime (sec) as a function of number of Co-Noise iterations.

(b) Runtime (sec) as a function of number of marginals.

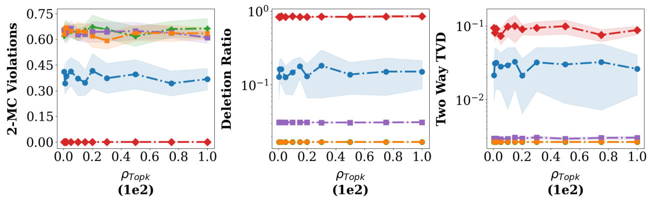
(c) Runtime (sec) for each dataset as a function of data size.

Figure 5: Runtime (sec) as a function of different parameters.

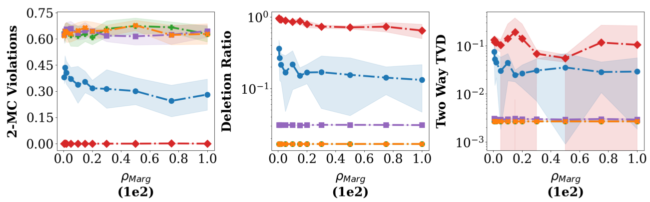
on Census, WVC-Vanilla takes about 82s, VC-approx-Baseline takes about 114s, Optimal-M 227s, Optimal-DC-Baseline 388s, and WVC-Dynamic 1245s. Figure 5b varies the number of 2-MCs. VC-approx-Baseline, WVC-Vanilla, and Optimal-DC-Baseline remain constant. Although the trend is noisy, Optimal-M shows an increase as hard constraints shrink the feasible region, though less sharply than WVC-Dynamic, indicating that DC violations dominate runtime (e.g., on COMPAS, Optimal-M can exceed WVC-Dynamic). WVC-Dynamic is typically highest due to many iterations balancing DC and 2-MC satisfaction. For Tax with 300 2-MCs, VC-approx-Baseline takes about 0.5s WVC-Vanilla, and Optimal-DC-Baseline take about 3s, Optimal-M 37s, and WVC-Dynamic 157s. In Figure 5c we see that WVC-Vanilla and VC-approx-Baseline scale linearly with data size (graphs are log scaled) and remain fastest. Optimal-DC-Baseline and Optimal-M generally occupies the middle tier. WVC-Dynamic is most expensive: larger datasets imply more potential conflict edges, extending iterative resolution. For Compas at 20K, WVC-Vanilla takes around 152s, VC-approx-Baseline take about 213s, Optimal-DC-Baseline 558s, Optimal-M 755s, and WVC-Dynamic 5772s.

6.5 Privacy Budget Effect

Figure 6 shows the sensitivity of our approaches and the baselines to the privacy budget for the Compas data, focusing on ρ_{Topk} (for selecting the top-k marginals) and ρ_{marg} (noisy threshold measurement). The trends for the rest of the datasets are similar (see [7] for the results on the rest of the datasets). Performance is largely robust to ρ_{Topk}, while increasing ρ_{marg} improves repair quality to a level where it stabilizes. Less accurate 2-MCs can also make



(a) Effect of ρ_{Topk} in Algorithm 2 on 2-MC violations, deletion ratio, and TVD for the Compas dataset.



(b) Effect of ρ_{marg} in Algorithm 2 on 2-MC violations, deletion ratio, and TVD for the Compas dataset.

Optimal-DC Optimal-M VC-approx WVC-Dynamic WVC-Vanilla

Figure 6: Effect of privacy parameters in Algorithm 2 on 2-MC violations, deletion ratio, and TVD for Compas.

satisfaction harder. WVC-Dynamic interpolates between the other algorithms and baselines. Optimal-DC-Baseline, WVC-Vanilla, and VC-approx-Baseline remain constant, as they optimize only DCs and are unaffected by privacy parameters (except for the results for 2-MC violations case where they vary due to the difference in chosen 2-MCs and the noisy thresholds). Optimal-M yields the largest deletion ratio and TVD since it strictly enforces the noisy 2-MCs (thus leading to their satisfaction). Figure 6a varies ρ_{Topk}. All methods exhibit nearly flat trends. For example, the standard deviation of the deletions across all the values ρ_{Topk} in WVC-Dynamic is around 2% deletions. Figure 6b varies ρ_{marg}. Optimal-M and WVC-Dynamic show decreasing deletion ratios as ρ_{marg} increases, since less noise moves thresholds closer to the private distribution and reduces required deletions. For WVC-Dynamic, as the accuracy of the threshold increases, the easier it is to satisfy the 2-MCs. Optimal-M and WVC-Dynamic show a decrease in TVD as the privacy budget increases, albeit non-monotonic. This is consistent with other aspects that improve with 2-MC threshold accuracy. However, our approach does not aim to optimize this aspect, so a monotonic and stable decrease is not guaranteed. For example, in the range between $\rho_{marg} = 0.5$ and $\rho_{marg} = 100$, the TVD after repairing with Optimal-M decreases by 0.03 and the deletion ratio of WVC-Dynamic decreases by 23%. Compared to the ground truth ($\rho_{marg}, \rho_{Topk} = \infty$), we observe a relatively significant difference in deletion ratios and TVD. For example, without noise WVC-Dynamic removes 2.6% of the data and the TVD is 0.0032 while with noise, WVC-Dynamic removes about 13% of the data and has TVD of 0.029. Overall, Algorithm 2 shows that the quality of the algorithm improves as ρ_{marg} increases. However, even $\rho_{marg} = 100$ is insufficient for ResidualPlanner [62] to compute an exact threshold in Algorithm 2, since it aims to optimize the global statistics for *an attribute pair* rather than the selected 2-way marginals. Exploring the effect of replacing ResidualPlanner with other mechanisms in Algorithm 2 is a subject for future work.

7 RELATED WORK

We next survey related work in the fields of DP synthetic data generation and data repair. Complementing these, our work is the first to explore *post-hoc repair of synthetic data generated from private data*, where the goal is not to generate synthetic data from scratch but rather to modify an already released synthetic dataset so that it becomes consistent with a given set of DCs while maintaining, or potentially improving, specific statistical attributes.

Private synthetic data generation. Releasing DP synthetic data enables analysts to work with a large, shareable dataset that supports arbitrary downstream computation while still providing formal privacy protection. A broad spectrum of techniques has been developed for generating such data [26, 27, 29, 32, 33, 40, 41, 56, 58, 59, 61, 63, 64]. These include methods that approximate the underlying distribution using low dimensional marginals [40, 41, 43, 64] as well as approaches based on GANs and other deep learning models [56, 58, 59, 63]. Empirical studies [57] find that marginal-based systems such as MST [40], PrivBayes [64], and AIM [56] offer some of the strongest utility in the private setting. Kamino [23] extends this line of work by generating DP synthetic data that additionally satisfies (possibly soft) DCs [8], allowing users to incorporate integrity requirements directly into the synthesis pipeline. Despite these advances, generating high-quality DP synthetic data remains computationally expensive, often requiring substantial runtime.

Data repair. Research on data repair spans a broad range of techniques aimed at restoring consistency with minimal modifications to the database [1, 3, 10, 20, 53]. Prior work has focused both on tuple deletions [1, 8, 25, 34, 37, 39, 47] and on value updates [3, 10, 12, 24, 28, 53], with the computational complexity of deletion-based repairs now relatively well-understood [37]. A recent line of work [34] has examined subset repairs under functional dependencies combined with representation constraints, which can be viewed as enforcing 1-way marginals. Our work generalizes this setting by considering the richer class of DCs [8] together with 2-way marginals, and by introducing both a dynamic repair strategy and a private procedure for obtaining 2-MCs.

8 CONCLUSION AND FUTURE WORK

We introduced synthetic data repair under DP, addressing the post-hoc enforcement of Denial Constraints and noisy 2-way marginals on existing synthetic datasets. We showed the problem is NP-hard and proposed ILP and weighted vertex cover repair methods. Experiments on real-world datasets demonstrate the tradeoff between reductions in DC and 2-MC violations and preserving statistical fidelity, downstream ML utility, and runtime, with the weighted approach offering a practical balance between quality and scalability.

Several directions are left for future exploration. First, extending the framework to support richer classes of statistical and fairness constraints beyond 2-MCs is a natural next step. Second, developing tighter theoretical bounds on the tradeoff between privacy noise, deletion ratio, and statistical fidelity would deepen understanding of the repair landscape. Finally, exploring the effect of different approaches for the prioritization of statistical constraints (varying utility functions) and release in Algorithm 2 may allow for a broader applicability of the framework.

REFERENCES

[1] Foto N. Afrati and Phokion G. Kolaitis. 2009. Repair Checking in Inconsistent Databases: Algorithms and Complexity. In *ICDT*. 31–41.

[2] Leopoldo Bertossi. 2011. *Database Repairing and Consistent Query Answering.* Morgan & Claypool Publishers.

[3] Leopoldo E. Bertossi, Solmaz Kolahi, and Laks V. S. Lakshmanan. 2013. Data Cleaning and Query Answering with Matching Dependencies and Matching Functions. *Theory Comput. Syst.* 52, 3 (2013), 441–482.

[4] Philip Bohannon, Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2007. Conditional functional dependencies for data cleaning. In *ICDE*.

[5] Mark Bun and Thomas Steinke. 2016. Concentrated differential privacy: Simplifications, extensions, and lower bounds. In *Theory of Cryptography Conference*. Springer, 635–658.

[6] Mark Cesar and Ryan Rogers. 2021. Bounding, concentrating, and truncating: Unifying privacy loss composition for data analytics. In *Algorithmic Learning Theory*. PMLR, 421–457.

[7] Itay Chairman, Xi He, and Amir Gilad. [n.d.]. Repairing Privately Generated Synthetic Data with Integrity and Statistical Constraints. https://github.com/ItayChairmanHuji/PrivateSyntheticDataRepair/blob/main/full_version.pdf.

[8] Jan Chomicki and Jerzy Marcinkowski. 2005. Minimal-change integrity maintenance using tuple deletions. *Inf. Comput.* 197, 1-2 (2005), 90–121.

[9] Xu Chu, Ihab F Ilyas, and Paolo Papotti. 2013. Discovering denial constraints. *Proceedings of the VLDB Endowment* 6, 13 (2013), 1498–1509.

[10] Xu Chu, Ihab F. Ilyas, and Paolo Papotti. 2013. Holistic data cleaning: Putting violations into context. In *ICDE*. 458–469.

[11] Kenneth L Clarkson. 1983. A modification of the greedy algorithm for vertex cover. *Inform. Process. Lett.* 16, 1 (1983), 23–25.

[12] Michele Dallachiesa, Amr Ebaid, Ahmed Eldawy, Ahmed K. Elmagarmid, Ihab F. Ilyas, Mourad Ouzzani, and Nan Tang. 2013. NADEEF: a commodity data cleaning system. In *SIGMOD*. 541–552.

[13] Jinshuo Dong, David Durfee, and Ryan Rogers. 2020. Optimal differential privacy composition for exponential mechanisms. In *International Conference on Machine Learning*. PMLR, 2597–2606.

[14] David Durfee and Ryan Rogers. 2021. One-shot DP Top-k mechanisms. DifferentialPrivacy.org. https://differentialprivacy.org/one-shot-top-k/.

[15] David Durfee and Ryan M. Rogers. 2019. Practical Differentially Private Top-k Selection with Pay-what-you-get Composition. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. 3527–3537. https://proceedings.neurips.cc/paper/2019/hash/b139e104214a08ae3f2ebcce149cdf6e-Abstract.html

[16] Cynthia Dwork. 2006. Differential Privacy. In *Automata, Languages and Programming*, Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener (Eds.). Springer Berlin Heidelberg.

[17] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating noise to sensitivity in private data analysis. In *Theory of cryptography conference*. Springer, 265–284.

[18] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam D. Smith. 2016. Calibrating Noise to Sensitivity in Private Data Analysis. *J. Priv. Confidentiality* 7, 3 (2016), 17–51. https://doi.org/10.29012/jpc.v7i3.405

[19] Cynthia Dwork, Aaron Roth, et al. 2014. The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.* 9, 3-4 (2014), 211–407.

[20] Ronald Fagin, Benny Kimelfeld, and Phokion G. Kolaitis. 2015. Dichotomies in the Complexity of Preferred Repairs. In *PODS*. 3–15.

[21] Sarah Flood, Miriam King, Renae Rodgers, Steven Ruggles, J. Robert Warren, and Michael Westberry. 2021. Integrated Public Use Microdata Series, Current Population Survey: Version 9.0 [dataset]. *Minneapolis, MN: IPUMS* (2021). https://doi.org/10.18128/D030.V9.0.

[22] Georgi Ganev, Bristena Oprisanu, and Emiliano De Cristofaro. 2022. Robin Hood and Matthew Effects: Differential Privacy Has Disparate Impact on Synthetic Data. In *ICML*, Vol. 162. 6944–6959.

[23] Chang Ge, Shubhankar Mohapatra, Xi He, and Ihab F. Ilyas. 2021. Kamino: Constraint-Aware Differentially Private Data Synthesis. *Proc. VLDB Endow.* 14, 10 (2021), 1886–1899.

[24] Floris Geerts, Giansalvatore Mecca, Paolo Papotti, and Donatello Santoro. 2013. The LLUNATIC Data-Cleaning Framework. *Proc. VLDB Endow.* 6, 9 (2013), 625–636.

[25] Amir Gilad, Daniel Deutch, and Sudeepa Roy. 2020. On Multiple Semantics for Declarative Database Repairs. In *SIGMOD*. 817–831.

[26] Moritz Hardt and Guy N. Rothblum. 2010. A Multiplicative Weights Mechanism for Privacy-Preserving Data Analysis. *2010 IEEE 51st Annual Symposium on Foundations of Computer Science* (2010), 61–70.

[27] Zhiqi Huang, Ryan McKenna, George Bissias, Gerome Miklau, Michael Hay, and Ashwin Machanavajjhala. 2019. PSynDB: Accurate and Accessible Private Data Generation. *Proc. VLDB Endow.* 12, 12 (2019), 1918–1921. https://doi.org/10.14778/3352063.3352099

[28] Solmaz Kolahi and Laks V. S. Lakshmanan. 2009. On Approximating Optimum Repairs for Functional Dependency Violations. In *Proceedings of the 12th International Conference on Database Theory* (St. Petersburg, Russia) *(ICDT '09)*. Association for Computing Machinery, New York, NY, USA, 53–62. https://doi.org/10.1145/1514894.1514901

[29] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. 2023. TabDDPM: Modelling Tabular Data with Diffusion Models. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research)*, Vol. 202. PMLR, 17564–17579. https://proceedings.mlr.press/v202/kotelnikov23a.html

[30] Nick Koudas, Avishek Saha, Divesh Srivastava, and Suresh Venkatasubramanian. 2009. Metric functional dependencies. In *ICDE*.

[31] M. Lichman. 2013. UCI machine learning repository. https://archive.ics.uci.edu/ml/datasets/adult.

[32] Terrance Liu, Jingwu Tang, Giuseppe Vietri, and Steven Wu. 2023. Generating Private Synthetic Data with Genetic Algorithms. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA (Proceedings of Machine Learning Research)*, Vol. 202. PMLR, 22009–22027. https://proceedings.mlr.press/v202/liu23ag.html

[33] Terrance Liu, Giuseppe Vietri, and Steven Wu. 2021. Iterative Methods for Private Synthetic Data: Unifying Framework and New Methods. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*. 690–702. https://proceedings.neurips.cc/paper/2021/hash/0678c572b0d5597d2d4a6b5bd135754c-Abstract.html

[34] Yuxi Liu, Fangzhu Shen, Kushagra Ghosh, Amir Gilad, Benny Kimelfeld, and Sudeepa Roy. 2024. The Cost of Representation by Subset Repairs. *Proc. VLDB Endow.* 18, 2 (2024), 475–487. https://www.vldb.org/pvldb/vol18/p475-liu.pdf

[35] Ester Livshits, Leopoldo E. Bertossi, Benny Kimelfeld, and Moshe Sebag. 2020. The Shapley Value of Tuples in Query Answering. In *ICDT*, Vol. 155. 20:1–20:19.

[36] Ester Livshits, Benny Kimelfeld, and Sudeepa Roy. 2018. Computing Optimal Repairs for Functional Dependencies. In *SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 225–237.

[37] Ester Livshits, Benny Kimelfeld, and Sudeepa Roy. 2020. Computing Optimal Repairs for Functional Dependencies. *ACM Trans. Database Syst.* 45, 1 (2020), 4:1–4:46. https://doi.org/10.1145/3360904

[38] Ester Livshits, Rina Kochirgan, Segev Tsur, Ihab F. Ilyas, Benny Kimelfeld, and Sudeepa Roy. 2021. Properties of Inconsistency Measures for Databases. In *SIGMOD*. 1182–1194.

[39] Andrei Lopatenko and Leopoldo E. Bertossi. 2007. Complexity of Consistent Query Answering in Databases Under Cardinality-Based and Incremental Repair Semantics. In *ICDT*. 179–193.

[40] Ryan McKenna, Gerome Miklau, and Daniel Sheldon. 2021. Winning the NIST Contest: A scalable and general approach to differentially private synthetic data. arXiv:2108.04978 [cs.CR] https://arxiv.org/abs/2108.04978

[41] Ryan McKenna, Brett Mullins, Daniel Sheldon, and Gerome Miklau. 2022. AIM: An Adaptive and Iterative Mechanism for Differentially Private Synthetic Data. *CoRR* abs/2201.12677 (2022). arXiv:2201.12677 https://arxiv.org/abs/2201.12677

[42] Ryan McKenna, Daniel Sheldon, and Gerome Miklau. 2019. Graphical-model based estimation and inference for differential privacy. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research)*, Vol. 97. PMLR, 4435–4444. http://proceedings.mlr.press/v97/mckenna19a.html

[43] Ryan McKenna, Daniel Sheldon, and Gerome Miklau. 2019. Graphical-model based estimation and inference for differential privacy. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA (Proceedings of Machine Learning Research)*, Vol. 97. PMLR, 4435–4444.

[44] Frank McSherry and Kunal Talwar. 2007. Mechanism Design via Differential Privacy. In *FOCS*. 94–103.

[45] Frank McSherry and Kunal Talwar. 2007. Mechanism design via differential privacy. In *48th Annual IEEE Symposium on Foundations of Computer Science (FOCS'07)*. IEEE, 94–103.

[46] Frank D McSherry. 2009. Privacy integrated queries: an extensible platform for privacy-preserving data analysis. In *SIGMOD*. 19–30.

[47] Dongjing Miao, Zhipeng Cai, Jianzhong Li, Xiangyu Gao, and Xianmin Liu. 2020. The computation of optimal subset repairs. *Proc. VLDB Endow.* 13, 12 (jul 2020), 2061–2074.

[48] Ilya Mironov. 2017. Rényi differential privacy. In *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*. IEEE, 263–275.

[49] Shubhankar Mohapatra, Amir Gilad, Xi He, and Benny Kimelfeld. 2025. Computing Inconsistency Measures Under Differential Privacy. *Proc. ACM Manag. Data* 3, 3 (2025), 140:1–140:27. https://doi.org/10.1145/3725397

[50] Francesco Parisi and John Grant. 2019. Inconsistency measures for relational databases. *arXiv preprint arXiv:1904.03403* (2019).

[51] ProPublica. [n.d.]. Compas recidivism risk score data and analysis. https://www.propublica.org/datastore/dataset/compas-recidivism-risk-score-data-and-analysis

[52] David Pujol, Amir Gilad, and Ashwin Machanavajjhala. 2023. PreFair: Privately Generating Justifiably Fair Synthetic Data. *Proc. VLDB Endow.* 16, 6 (2023), 1573–1586. https://doi.org/10.14778/3583140.3583168

[53] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic Data Repairs with Probabilistic Inference. *PVLDB* 10, 11 (2017), 1190–1201.

[54] Ryan Rogers and Thomas Steinke. 2021. A Better Privacy Analysis of the Exponential Mechanism. DifferentialPrivacy.org. https://differentialprivacy.org/exponential-mechanism-bounded-range/.

[55] Lucas Rosenblatt, Xiaoyan Liu, Samira Pouyanfar, Eduardo de Leon, Anuj Desai, and Joshua Allen. 2020. Differentially private synthetic data: Applied evaluations and enhancements. *arXiv preprint arXiv:2011.05537* (2020).

[56] Lucas Rosenblatt, Xiaoyan Liu, Samira Pouyanfar, Eduardo de Leon, Anuj Desai, and Joshua Allen. 2020. Differentially Private Synthetic Data: Applied Evaluations and Enhancements. *CoRR* abs/2011.05537 (2020). arXiv:2011.05537 https://arxiv.org/abs/2011.05537

[57] Yuchao Tao, Ryan McKenna, Michael Hay, Ashwin Machanavajjhala, and Gerome Miklau. 2021. Benchmarking Differentially Private Synthetic Data Generation Algorithms. *CoRR* abs/2112.09238 (2021).

[58] Reihaneh Torkzadehmahani, Peter Kairouz, and Benedict Paten. 2019. DP-CGAN: Differentially Private Synthetic Data and Label Generation. In *CVPR*. 98–104.

[59] Reihaneh Torkzadehmahani, Peter Kairouz, and Benedict Paten. 2020. DP-CGAN: Differentially Private Synthetic Data and Label Generation. arXiv:2001.09700 [cs.LG]

[60] Vijay V Vazirani. 1997. Approximation algorithms. *Georgia Inst. Tech* (1997).

[61] Giuseppe Vietri, Cédric Archambeau, Sergül Aydöre, William Brown, Michael Kearns, Aaron Roth, Amaresh Ankit Siva, Shuai Tang, and Zhiwei Steven Wu. 2022. Private Synthetic Data for Multitask Learning and Marginal Queries. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022.* http://papers.nips.cc/paper_files/paper/2022/hash/7428310c0f97f1c6bb2ef1be99c1ec2a-Abstract-Conference.html

[62] Yingtai Xiao, Guanlin He, Danfeng Zhang, and Daniel Kifer. 2023. An Optimal and Scalable Matrix Mechanism for Noisy Marginals under Convex Loss Functions. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023.* http://papers.nips.cc/paper_files/paper/2023/hash/414f4c9fe9653e5de98fad6964d50315-Abstract-Conference.html

[63] Jinsung Yoon, James Jordon, and Mihaela van der Schaar. 2019. PATE-GAN: Generating Synthetic Data with Differential Privacy Guarantees. In *International Conference on Learning Representations.* https://openreview.net/forum?id=S1zk9iRqF7

[64] Jun Zhang, Graham Cormode, Cecilia M Procopiuc, Divesh Srivastava, and Xiaokui Xiao. 2017. Privbayes: Private data release via bayesian networks. *ACM Transactions on Database Systems (TODS)* 42, 4 (2017), 1–41.

[65] Zhikun Zhang, Tianhao Wang, Ninghui Li, Jean Honorio, Michael Backes, Shibo He, Jiming Chen, and Yang Zhang. 2021. PrivSyn: Differentially Private Data Synthesis. In *USENIX*. 929–946.